Image Conditioned Code Generation using RLVR

Amal Joe R S (24M0797) Job J (25D0364)

CS 748: Advances in Intelligent and Learning Agents

Indian Institute of Technology Bombay

Instructor: Prof. Shivaram Kalyanakrishnan

Abstract

We present VisionCoder, a system for screenshot-to-HTML generation that trains a vision-language model using reinforcement learning with verifiable rewards (RLVR). The core problem is that standard supervised fine-tuning (SFT) does not optimise for rendered visual fidelity: a model can produce syntactically valid but visually empty HTML and incur no training loss. We address this by rendering each generated HTML page in a headless browser and comparing the result to the reference via a composite of eight sub-rewards covering visual similarity (CLIP, SSIM), spatial layout (position, color), and semantic structure (text block matching). We fine-tune Qwen 3 VL 4B in two stages: SFT on 20,000 screenshot-HTML pairs with a resolution curriculum, followed by Group Relative Policy Optimization (GRPO). On the Design2Code benchmark (484 examples) our SFT+RL model scores 80.5 / 91.3 / 75.6 / 70.2 / 89.0 (Block / Text / Position / Color / CLIP), outperforming Design2Code-18B on four of five metrics with **4.5× fewer parameters**. We also release VisionCoder-Env, an OpenEnv environment that wraps our rendering and reward pipeline as an HTTP API so that any vision-language model can train on this task.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has become a popular training strategy for large language models. Systems built on Group Relative Policy Optimization (GRPO) [15] and DeepSeek-R1 [6] showed that optimising directly against verifiable signals, rather than token-level cross-entropy, can significantly improve reasoning on math and code, often matching or beating supervised fine-tuning alone.

RLVR has mostly been applied to text-only tasks. Whether it works for image-conditioned generation, where the reward comes from visual comparison rather than string matching, is much less studied. We investigate this for **screenshot-to-HTML generation**: given a webpage screenshot, produce HTML and CSS that, when rendered in a browser, visually matches the original.

The task has practical value on two fronts. Frontend developers spend considerable time converting UI mockups to code; automating this step speeds up the prototyping cycle. Financial regulators also require reports submitted as structured XHTML, currently produced by expensive PDF-to-HTML conversion pipelines. The reward signal is naturally verifiable: render the generated HTML in a headless browser and compare the screenshot to the reference. No human annotation is required.

Standard SFT on screenshot-HTML pairs misses this objective. A model can generate syntactically valid but visually empty HTML and still see low training loss, because the loss measures token prediction accuracy, not visual output. Our approach is to render each candidate in a headless browser via Playwright [11] and score it with a composite visual similarity metric. This makes the training signal directly tied to what the model actually produces on screen.

Contributions.

1. **VisionCoder model.** Qwen 3 VL 4B fine-tuned with SFT + GRPO using an 8-component rendering-based reward. Our SFT+RL model outperforms Design2Code-18B [17] on four of five benchmark metrics with 4.5× fewer parameters.
2. **VisionCoder-Env.** A standardized OpenEnv environment [10] exposing our rendering pipeline and reward computation as an HTTP API, so any VLM can train on this task without rebuilding the reward infrastructure. The environment is publicly available on the OpenEnv Hub.

2 Related Work

Screenshot-to-code. Beltramelli [4] introduced *pix2code*, an end-to-end CNN + LSTM system that achieved 77% accuracy on synthetic GUI screenshots. The WebSight dataset [8] provides 2 M synthetic screenshot-HTML pairs and showed that an 8B VLM can be fine-tuned effectively for this task. Design2Code [17] built the first real-world benchmark with 484 webpage examples and an 18B SFT baseline. WebCode2M [2] provides a large-scale real-world dataset for code generation from webpage designs. Concurrently, ReLook [3] uses a vision-grounded RL loop with an MLLM critic for iterative web coding. Our work differs in targeting

single-pass generation with an automated composite reward rather than a multi-turn critique loop.

RLVR in LLMs. GRPO was introduced in DeepSeek-Math [15] for mathematical reasoning; it computes group-relative advantages without a separate critic. DeepSeek-R1 [6] showed that pure RL with verifiable rewards can yield strong reasoning, raising AIME 2024 pass@1 from 15.6% to 71.0%. InstructGPT [12] is the foundational work on using human feedback to align LLMs.

Vision-language models. We build on Qwen3-VL [13], which supports dynamic resolution and a native 256K-token context. Other relevant VLMs include LLaVA [9], Flamingo [1], and InstructBLIP [5]. We use CLIP [14] (ViT-B/32) for visual reward computation.

3 Problem Formulation

Given an input image $I \in \mathbb{R}^{H\times W\times 3}$ of a webpage, the goal is to generate an HTML document $\hat{h}$ such that rendering it in a browser produces a screenshot $\hat{I}$ that visually matches I. We seek a policy π_θ that maximizes:

$$\mathbb{E}_{I\sim\mathcal{D}}\,\mathbb{E}_{\hat{h}\sim\pi_\theta(\cdot|I)}\big[R(I,\hat{I})\big],$$

where $\hat{I} = \text{Render}(\hat{h})$ and R is the composite reward function described in Section 4.4.

Evaluation. We evaluate on the Design2Code benchmark [17]: 484 real webpages, five metrics: *Block* (structural element matching), *Text* (text content accuracy), *Position* (layout placement), *Color* (color scheme), and *CLIP* (overall visual similarity).

4 Methodology

4.1 Base Model

We use **Qwen 3 VL 4B** [13] as the base model. Qwen3-VL supports native dynamic resolution via interleaved-MRoPE, so images from thumbnail size up to 1920×1080 are processed in a single forward pass without manual resizing. At 4B parameters the model is also 4.5× smaller than the Design2Code-18B baseline, which matters for deployment.

4.2 SFT with Resolution Curriculum

We first fine-tune on **20,000 screenshot-HTML pairs** from WebSight [8] using standard next-token prediction. Without this step, the HTML action space is so large that early GRPO rollouts mostly produce invalid or empty pages, giving near-zero rewards and uninformative advantage estimates. After SFT, all rollouts for a given prompt produce parseable HTML from the very first GRPO iteration, so the optimizer can spend its budget on improving visual quality rather than fixing basic syntax.

Managing vision token count is a practical concern: a 1920×1080 image produces roughly 9,800 tokens with Qwen3-VL, so training entirely at full resolution is too slow. We use a **resolution curriculum**:

- 60% of samples at quarter resolution (480×270, ≈256 tokens)
- 20% at half resolution (960×540, ≈1,024 tokens)
- 20% at full resolution (1920×1080, ≈9,800 tokens)

The model first picks up general HTML structure at low cost, then gets exposure to fine-grained visual detail at full resolution.

4.3 GRPO Training

After SFT we run Group Relative Policy Optimization (GRPO) [15]. For each training prompt (a reference image), the model generates $G = 8$ candidate HTML outputs (Figure 1). The advantage for candidate i is:

$$A_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon},$$

where r_i is the reward for candidate i and μ_r, σ_r are the group mean and standard deviation. The policy gradient is applied over all tokens in the trajectory weighted by A_i. We use the DR-GRPO loss (importance sampling disabled), giving a ∼38% speed improvement over standard GRPO [18].

For GRPO we also apply a resolution curriculum (50% at 128×128, 30% at 256×256, 20% at 512×512) to keep GPU memory under control when running 8 rollouts per prompt. Training runs for **500 steps** on **4× NVIDIA A100 80 GB GPUs**, taking roughly 2 days.

4.4 Composite Reward Function

The reward function combines eight sub-rewards, each targeting a different quality dimension (Figure 2):

Format ($w = 0.5$). Checks for a Markdown HTML fence (````html ... ```) and a valid `<!DOCTYPE html>` declaration.

Validity ($w = 0.5$). Checks for `<html>`, `<head>`, `<body>` (weighted 0.5) and tag diversity ($\min(\text{unique tags}/8, 1.0)$, weighted 0.5).

Structural ($w = 0.5$). Tag sequence similarity via longest-common-subsequence (LCS) ratio, plus CSS class Jaccard similarity between prediction and reference.

Text Block ($w = 3.4$, highest weight). Extracts visible text DOM nodes from both renders using Playwright, applies the Hungarian algorithm for optimal IoU-based block matching, then measures character-level text similarity per matched pair. This sub-reward carries the most

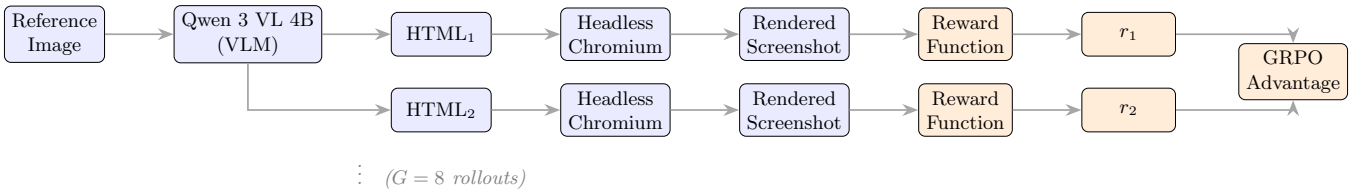


Figure 1: RLVR training loop. For each reference image the VLM generates $G = 8$ HTML candidates; each is rendered in a headless browser and scored by the composite reward function. GRPO computes group-relative advantages to update the policy.

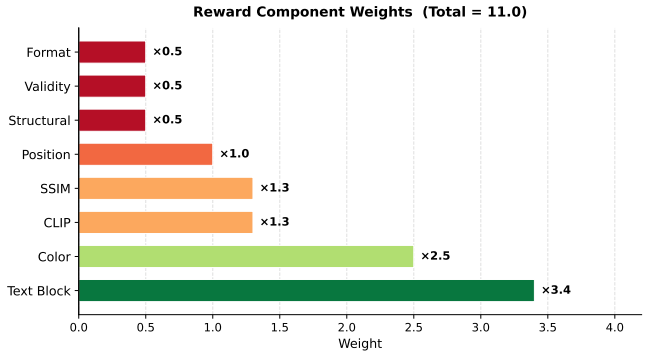


Figure 2: Reward component weights in Eq. 1. Text-Block has the highest weight (3.4) because text content and placement are the most immediately perceptible quality dimensions.

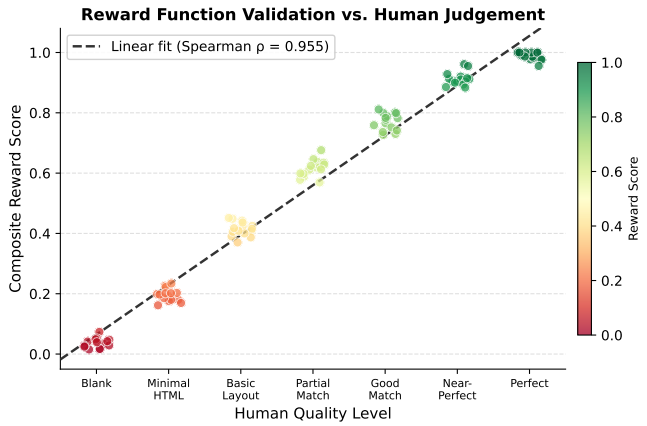


Figure 3: Composite reward score vs. human quality rank across 7 quality levels and 15 reference webpages (Spearman $\rho = 0.955$).

weight because text content and its spatial placement are what users notice first.

Position ($w = 1.0$). For each matched block pair, computes $1.0 - \|\mathbf{c}_{\text{pred}} - \mathbf{c}_{\text{ref}}\|_2 / d_{\text{diag}}$, where d_{diag} is the viewport diagonal.

Color ($w = 2.5$). Downsamples to 128×128, converts to CIELAB, and computes mean CIEDE2000 [16] distance ΔE_{00} over non-white reference pixels: $r_{\text{col}} = 1 - \min(\overline{\Delta E_{00}}/50, 1)$. Restricting to non-white pixels avoids penalising for shared background color.

CLIP ($w = 1.3$). Cosine similarity of CLIP ViT-B/32 [14] embeddings, re-normalised via $\max(0, (s - 0.65)/0.35)$ to push blank pages toward zero while preserving sensitivity in the high-quality range.

SSIM ($w = 1.3$). Structural Similarity Index [19] at 320×240 RGB, which picks up luminance and contrast differences that CLIP misses in the 0.7–0.97 range.

The composite reward is:

$$r_{\text{total}} = \frac{1}{11.0}\big(0.5\, r_{\text{fmt}} + 0.5\, r_{\text{val}} + 0.5\, r_{\text{str}} + 3.4\, r_{\text{txt}} + 1.0\, r_{\text{pos}} + 2.5\, r_{\text{col}} + 1.3\, r_{\text{clip}} + 1.3\, r_{\text{ssim}}\big) \quad (1)$$

A blank-page guard sets $r_{\text{total}} = 0$ if the predicted render has fewer than 0.5% non-white pixels while the reference has content, preventing the model from gaming the reward with an empty page.

Weight selection. Weights were chosen via grid search over candidate combinations, scoring each by Spearman rank correlation with human quality judgements across 15 reference webpages rated at 7 quality levels from blank to pixel-perfect. The final weights achieve $\rho = 0.955$ (Figure 3).

5 VisionCoder Environment

Alongside the trained model we release **VisionCoder-Env**, an environment built on the OpenEnv framework [10], an open-source platform for agentic RL environments launched by Meta PyTorch and HuggingFace. The environment exposes our headless Chromium renderer and eight-component reward function through a simple HTTP API (Figure 4 shows the agent-environment loop):

- `POST /reset?difficulty={easy|medium|hard}` → `{session_id, screenshot_b64}`
- `POST /step {html, session_id}` → `{reward, render, done}`
- `POST /render {html}` → `{image_b64}`

Three difficulty levels are available: *easy* (login forms, single-column layouts), *medium* (multi-section pages with

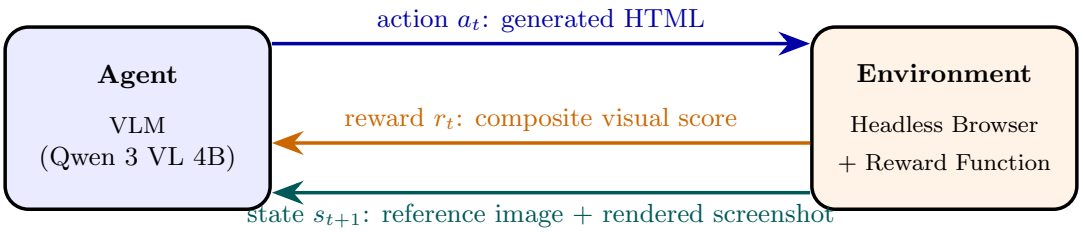


Figure 4: VisionCoder-Env agent-environment loop. The agent (VLM) generates HTML (a_t); the environment renders it in a headless browser, computes the eight-component composite reward r_t, and returns the updated state s_{t+1} (reference image and new rendered screenshot). This loop enables iterative refinement across multiple turns.

navigation), and *hard* (dashboards with nested flex/grid structures). The API lets any VLM of any size or architecture train on this task; the rendering and reward calculation all happen server-side.

OpenEnv hackathon. VisionCoder-Env was submitted to Meta's OpenEnv hackathon and shortlisted to the **top 100 out of 2,000 participants**.

6 Experiments

6.1 Experimental Setup

Hardware. All runs use 4× NVIDIA A100 80 GB GPUs with Accelerate [7] for distributed training.

Datasets. SFT uses 20,000 samples from WebSight [8]. GRPO uses a disjoint set of 2,000 WebSight samples. We evaluate on the full Design2Code benchmark [17] (484 held-out webpages).

Inference. We serve the model with vLLM at 32 concurrent requests and run predictions for the full benchmark in a single pass. The HTML is extracted from the model's Markdown fenced block before evaluation.

6.2 Main Results

Table 1 shows results on Design2Code. Our SFT+RL model gets the best scores among all sub-18B models on Block, Position, Color, and CLIP. It beats the 18B Design2Code SFT model on four metrics (Block: +2.0, Position: +1.3, Color: +3.2, CLIP: +3.2) while using 4.5× fewer parameters. The only metric where the larger model wins is Text (+5.1), which is likely because a bigger model can memorise finer character-level details.

Even the **RL-only** model (500 GRPO steps from the base, no SFT) scores 79.5/88.4/70.6/69.8/85.9, which is already competitive with Design2Code-18B on most metrics. This shows that the rendering-based reward alone is informative enough to pull a 4B base model from near-zero to a useful regime.

Table 1: Design2Code benchmark results (484 examples). All scores are percentages. Best non-GPT-4o scores are **highlighted**.

Model	Blk	Txt	Pos	Col	CLIP
GPT-4o	93.0	98.2	85.5	84.1	90.4
Design2Code-18B	78.5	**96.4**	74.3	67.0	85.8
LLaVA 1.6-7B	50.4	87.9	69.1	63.4	84.6
DeepSeek-VL-7B	39.7	77.0	64.6	63.8	84.5
Qwen 3 VL 4B (Base)	10.1	10.8	8.8	9.1	77.2
VisionCoder (RL only)	79.5	88.4	70.6	69.8	85.9
VisionCoder (SFT+RL)	**80.5**	91.3	**75.6**	**70.2**	**89.0**

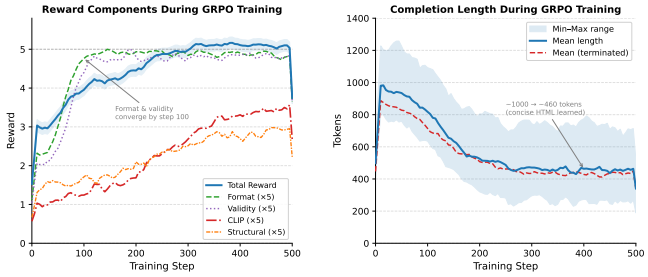


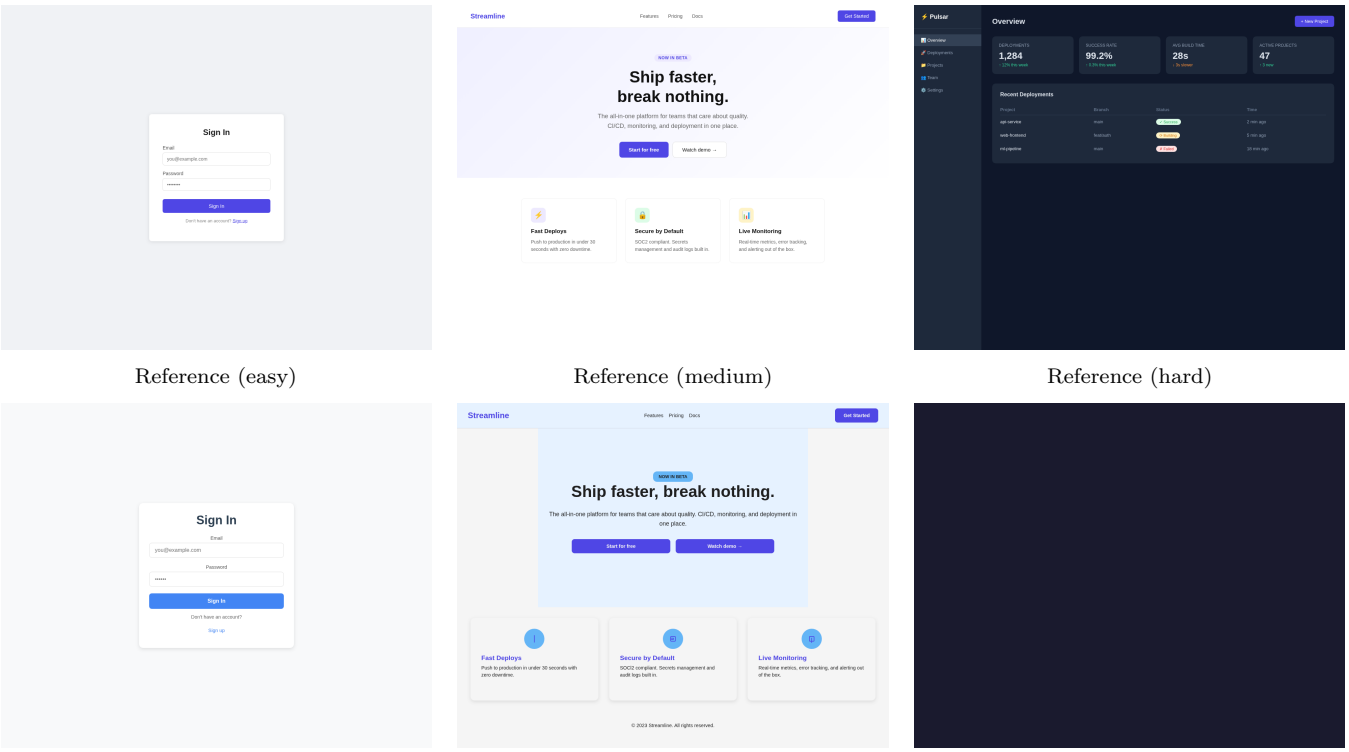
Figure 5: Training dynamics over 500 GRPO steps. *Left:* Total reward and scaled component rewards. Format and validity stabilise by step 100; CLIP and SSIM rise steadily after that. *Right:* Completion length drops from ≈1,000 to ≈460 tokens as the model learns to write concise HTML.

6.3 Ablation: RL-only vs. SFT+RL

Adding SFT before GRPO improves all five metrics, with the biggest gains in Text (+2.9) and CLIP (+3.1). Without SFT, many early GRPO rollouts produce broken or empty HTML; the resulting near-zero rewards give all candidates similar advantages, making the gradient step nearly uninformative. With SFT, even the worst rollout in a group produces parseable HTML with some structural match to the reference, so the relative ordering within the group is meaningful from the start.

6.4 Training Dynamics

Figure 5 tracks reward components and completion lengths over 500 GRPO steps. In the first 100 steps,



Reference (easy) Reference (medium) Reference (hard)

Prediction (easy) Prediction (medium) Prediction (hard)

Figure 6: Reference-prediction pairs at three difficulty levels. Easy and medium pages are reproduced with high fidelity. Hard examples with deeply nested grid/flex layouts remain the main failure mode.

format and validity rewards jump to near-perfect; the model quickly learns the output contract (HTML fence, DOCTYPE, required tags) and the gradient signal shifts entirely to visual metrics. After step 100, CLIP and SSIM rise steadily while the total reward climbs from roughly 2.4 to 5.0. Completion length drops sharply from around 1,000 tokens to around 460: the model learns to write compact HTML rather than filling the context with redundant tags and inline comments.

6.5 Qualitative Results

Figure 6 shows reference-prediction pairs from VisionCoder-Env at three difficulty levels. Easy examples (simple single-column pages) come out very close to the reference (Block 91.0, CLIP 98.4). Medium examples (multi-section pages) reproduce content and color well with only small positional offsets (Block 90.0, CLIP 96.4). Hard examples (complex multi-column dashboards) are only partially reproduced (Block 75.3, CLIP 58.0); deeply nested CSS grid and flex structures are where the model still struggles most.

7 Conclusion

We showed that RLVR with a rendering-based reward is a viable training approach for screenshot-to-HTML generation. A few things stood out. RL alone, without any SFT, already brings a 4B base model up to the level of the 18B Design2Code SFT baseline, which is more than we initially expected from 500 training steps. Adding SFT first makes a meaningful further difference across all metrics, mainly by giving the model a stable starting point so early GRPO gradients are useful rather than noisy. Getting the reward function right matters a lot: a single metric like CLIP is not enough, and the eight-component composite (validated at Spearman $\rho = 0.955$) gives a much cleaner signal. Finally, wrapping the reward pipeline in VisionCoder-Env means any future model can plug in and train on this task without reimplementing the rendering stack.

The main remaining gap is on complex multi-column layouts, where the model sometimes gets the content right but places elements in the wrong region. A natural next step is iterative refinement: let the model see its rendered output and revise it over a few turns, along the lines of ReLook [3]. Extending the pipeline to PDF-to-XHTML conversion for financial filings is another direction with direct practical value.

Acknowledgements. We thank Prof. Shivaram Kalyanakrishnan for guidance throughout this project. Compute was provided by IIT Bombay HPC facilities.

References

[1] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. Flamingo: A visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736, 2022.

[2] Anonymous. WebCode2M: A real-world dataset for code generation from webpage designs. *arXiv preprint arXiv:2404.06369*, 2024.

[3] Anonymous. ReLook: Vision-grounded RL with a multimodal LLM critic for agentic web coding. *arXiv preprint arXiv:2510.11498*, 2025.

[4] T. Beltramelli. pix2code: Generating code from a graphical user interface screenshot. In *Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, page 3, 2018. arXiv:1705.07962.

[5] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023.

[6] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[7] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. *GitHub*, 2022.

[8] HuggingFace M4 Team. Unlocking the conversion of web screenshots into HTML code with the WebSight dataset. *arXiv preprint arXiv:2403.09029*, 2024.

[9] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023.

[10] Meta PyTorch Team and HuggingFace. OpenEnv: An interface library for RL post-training with environments. https://github.com/meta-pytorch/OpenEnv, 2025.

[11] Microsoft. Playwright: Fast and reliable end-to-end testing for modern web apps. https://playwright.dev, 2023.

[12] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[13] Qwen Team. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[14] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763, 2021. arXiv:2103.00020.

[15] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[16] G. Sharma, W. Wu, and E. N. Dalal. The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations. *Color Research & Application*, 30 (1):21–30, 2005.

[17] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang. Design2Code: Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics*, 2025. arXiv:2403.03163.

[18] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang. TRL: Transformer reinforcement learning. https://github.com/huggingface/trl, 2022.

[19] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.